

Week 3, Day 1 — presenter notes

Slides: open [week-03-day-01.html](#) in Chrome/Firefox → **F** fullscreen → arrows. **N** shows the speaker note for the current slide. **P** prints to PDF.

This session is **control flow + collections + survival functions**. No SMS code. Two labs. Dense — protect the backing-array demo and comma-ok.

Timing (3–3.5 hours)

Clock	Block	Slides
0:00–0:15	Homework check (<code>FullName</code> + exercises)	1–2
0:15–0:25	Outcomes + SE framing	3–4
0:25–0:55	<code>if</code> , short statement, <code>switch</code>	5–8
0:55–1:20	<code>for</code> , <code>range</code> , <code>break/continue</code> (live <code>control.go</code>)	9–12
1:20–2:00	Arrays/slices, <code>append</code> , backing gotcha (live)	13–17
2:00–2:25	Maps, comma-ok, freq (live)	18–21
2:25–2:40	Functions survival kit + test tables	22–24
2:40–end	Lab 1 then Lab 2	25–end

There is **no agenda slide** and **no homework slide**. Homework stays on the student handout. If late, cut variadic/closures live typing — never cut the slice gotcha or comma-ok.

Live demo script

1. Control (`~/epic-go/week03/control.go`)

```
package main

import "fmt"

func main() {
    grade := "B"
    switch grade {
    case "A", "B":
        fmt.Println("good")
    }
```

```
default:
    fmt.Println("work")
}
for i, r := range "Go" {
    fmt.Printf("%d %c\n", i, r)
}
}
```

```
gofmt -w control.go
go run control.go
```

2. Slice gotcha (`~/epic-go/week03/slices.go`)

```
package main

import "fmt"

func main() {
    a := []int{10, 20, 30, 40}
    b := a[:2]
    fmt.Println("before", a, b, "cap(b)=", cap(b))
    b = append(b, 99)
    fmt.Println("after ", a, b)
}
```

Draw the backing array. Then show a full-capacity case where `append` allocates and `a` is unchanged.

3. Frequency (`~/epic-go/week03/freq.go`)

```
words := []string{"SE", "CS", "SE", "IT", "SE"}
count := make(map[string]int)
for _, w := range words {
    count[w]++
}
```

Point out: missing key reads as `0`, so `++` works. Then comma-ok for `"SE"`.

4. Lab package (`~/epic-go/week03/slicekit/`)

```
mkdir -p ~/epic-go/week03/slicekit
cd ~/epic-go/week03/slicekit
go mod init epiclearn/week03
```

Students implement `Contains`, `Unique`, `Freq` with tests. Fail a test first when you demo `Contains`.

Lab gate (they do not leave without)

- [] Worksheet filled (control, slice gotcha, comma-ok)
- [] Paper test table for `Contains`
- [] go test green for `Contains`, `Unique`, `Freq`
- [] Can explain the shared backing array in one sentence
- [] GitHub URL collected

Homework

Slice/map exercises on the handout; harden `Unique` / `Freq`; optional Effective Go reading (slices + maps sections).

Typical failures

Symptom	Fix
<code>if score {</code>	need a bool: <code>if score != 0</code>
missing braces on <code>if</code>	braces required in Go
<code>append(s, x)</code> without assign	<code>s = append(s, x)</code>
slice overwrite surprise	draw backing array; copy if independent
<code>m[k] = v</code> on nil map	<code>m = make(map[K]V)</code> first
test fails on <code>fmt.Sprintf(map)</code>	assert keys with comma-ok, not print order
<code>Unique</code> loses order	append to result only on first see
unused loop variable	use <code>_</code>
<code>go test: no go.mod</code>	<code>go mod init epiclearn/week03</code>

What not to teach today

Methods, pointers as theory, modules as theory, REST, Gin, the SMS repo, `defer`, generics, channels. Tease week 4 only.